Subsystem 5 Forum Database Guide

Oct 20, 2026 · @Michael Pieterse

Overview

`subsystem5_forum_oracle.sql` builds the full Community Forum database in Oracle. One run creates 11 tables, their keys and constraints, indexes, three views, and the six forum categories from the SOW.

The script has no triggers. The database only guards structure: valid keys, valid values, and correct relationships. Anything that is behavior, like updating a vote count, belongs in our Java service code. Section "Who enforces what" below lists exactly which is which.

One table is not ours. `APP_USER` is a local stand-in for the Shared Core user table, so our foreign keys have something to point at during development. At integration, Shared Core owns that table and we stop creating it.

The script is split into numbered sections:

Section	What it does	Keep for integration?
0. Clean slate	Drops our tables if they exist, so the script can re-run	Dev only
1. APP_USER	Stand-in user table	No, Shared Core replaces it
2. Forum tables	Creates the 10 forum tables	Yes
3. Late constraint	Adds the helpful-reply foreign key	Yes
4. Indexes	Speeds up the queries the API runs	Yes
5. App-layer rules	Comment block only, no code	Reference
6. Views	Unanswered, trending, moderation queue	Yes
7. Seed data	The six SOW categories	Yes
8. Sample data + tests	Test rows and smoke-test queries	No, delete it

Running the script

Run the whole file as a script with F5. Ctrl+Enter only runs one statement and will break on the `/` lines.

1. Open Oracle SQL Developer and connect to your database (Oracle 19c, 21c XE, or 23ai all work).
2. Use a schema you own, not SYS or SYSTEM. The clean-slate step drops tables by name.
3. Open `subsystem5_forum_oracle.sql` in a SQL worksheet.
4. Press F5 (Run Script).
5. Check the Script Output tab. Every statement should say created, inserted, or completed. No ORA errors.
6. Scroll to the bottom for the smoke-test query results.

Re-running is safe. Section 0 drops all 11 tables first, so you always start clean. That also means re-running wipes any data you added by hand.

The `/` on its own line ends a PL/SQL block. Only section 0 uses one now. Don't delete it.

The tables

Everything hangs off POST. A post belongs to one category and one author, and replies, tags, votes, and flags all point back to it.

Table	What one row is	Primary key	Points to	Notes
APP_USER	A platform user	id	nothing	Stand-in for Shared Core. Roles: STUDENT, MENTOR, STAFF, ADMIN
CATEGORY	One forum category	id (auto number)	nothing	Name is unique. `is_active = 0` retires it without deleting posts
POST	A thread	id (UUID)	APP_USER, CATEGORY, REPLY (helpful)	Holds `vote_score`, `reply_count`, `last_activity_at` counters

Table	What one row is	Primary key	Points to	Notes
REPLY	A reply to a post	id (UUID)	POST, APP_USER, REPLY (parent)	`parent_reply_id` is null for a top-level reply, set for a nested one
TAG	A label like "Capital One"	id (auto number)	nothing	Type is employer, role, or other
POST_TAG	One tag on one post	post_id + tag_id	POST, TAG	Bridge table for many-to-many
POST_VOTE	One user's upvote on a post	post_id + user_id	POST, APP_USER	Two-column key means one vote per user per post
REPLY_VOTE	One user's upvote on a reply	reply_id + user_id	REPLY, APP_USER	Same rule for replies
FLAG	A report on a post or reply	id (UUID)	POST, REPLY, APP_USER	`reply_id` null means the whole post is flagged. Status: open, actioned, dismissed
MODERATION_ACTION	A moderator's decision on a flag	id (UUID)	FLAG, APP_USER	One action per flag. Action: remove, warn, dismiss
FORUM_USER_SETTINGS	Forum-only settings for a user	user_id	APP_USER	`hide_post_history` and `reputation`

A few conventions run through every table:

- **IDs are text UUIDs.** `VARCHAR2(36)`, like `3f2a9c1e-5b7d-4e8a-9c21-7d4e5f6a8b90`. In Java, use `UUID.randomUUID().toString()` or leave it out and let Oracle generate one.
- **True/false is 0 or 1.** Columns like `is_pinned` are `NUMBER(1)`. Map them to `boolean` in Java.

- **Nothing is hard-deleted.** Removing a post or reply sets `status = 'removed'`. Always filter on `status = 'visible'` when showing content.
- **Long text is CLOB.** `post.body` and `reply.body`. Read them with `getString()` in JDBC.
- **Timestamps default to now.** You don't need to set `created_at` on insert.

Who enforces what

If a rule is in the left column, the database rejects bad data for you. If it's in the right column, our Java code has to do it, or it won't happen at all.

Database enforces	Java must enforce
One vote per user per post or reply	Blocking users from upvoting their own post or reply
Helpful reply must be a reply on the same post	Bumping `post.reply_count` and `last_activity_at` when a reply is added
A nested reply's parent must be in the same post	Lowering `reply_count` when moderation removes a reply
A flagged reply must belong to the flag's post	Updating `vote_score` on every vote added or removed
A pinned post must record `pinned_by`	Checking the pinner is STAFF (through Shared Core RBAC)
Only one moderation action per flag	Refusing to act on a flag that isn't open
Valid values for status, role, tag type, action	Setting the flag's status after a moderation action
Category and tag names are unique	Soft-deleting the post or reply when the action is remove

The rule that matters most

Update counters in the same transaction as the insert that causes them, and let SQL do the math.

```
-- Right: Oracle adds 1 to whatever the current value is
UPDATE post SET reply_count = reply_count + 1, last_activity_at =
SYSTIMESTAMP
 WHERE id = ?;
```

Don't read the count into Java, add 1, and write it back. If two students reply at the same moment, both read 4, both write 5, and one reply goes uncounted.

In JDBC, that looks like this:

1. `conn.setAutoCommit(false)`
2. Insert the reply.
3. Run the counter update above.
4. `conn.commit()`, or `conn.rollback()` if either step throws.

Section 5 of the script and the sample data in section 8 show the exact statements for every case: replies, votes, and moderation.

Views and indexes

Views are saved queries. You select from them like a table, so the API doesn't repeat the same filters everywhere.

View	Feeds	What it returns
`v_unanswered_posts`	GET /posts?sort=unanswered	Visible posts with `reply_count = 0`
`v_trending_posts`	Trending tab	Visible posts active in the last 7 days, with `trend_score = reply_count x 2 + vote_score`
`v_moderation_queue`	GET /moderation/queue	Open flags with the post title and who flagged it

Add your own `ORDER BY` and pagination when you query a view. Example for page 2, 20 per page:

```
SELECT * FROM v_trending_posts
 ORDER BY trend_score DESC
 OFFSET 20 ROWS FETCH NEXT 20 ROWS ONLY;
```

The views don't check roles. The moderation queue must only be reachable by STAFF and ADMIN, and that check lives in the API.

Indexes make the common filters fast. Oracle doesn't index foreign keys on its own, so section 4 adds them for the columns the API filters on: category, author, post, status, and created date. If you add a new query that filters on a column with no index, tell the team before adding one.

Full-text search through Oracle Text is in section 4 but commented out. It needs the `CTXAPP` role. Until we have that, search with `UPPER(title) LIKE UPPER('%term%')`.

Testing

Section 8 loads four users, two posts, two replies, votes, and one flag, then runs the smoke-test queries. The sample IDs are fixed and easy to spot: users start with `0000`, posts with `1000`, replies with `2000`, flags with `3000`.

After F5, the results should match this:

Check	Expected
Post 1 vote_score	1
Post 1 reply_count	1 (two replies, one removed by moderation)
Post 1 is_pinned	1, with a helpful reply set
Post 2	Shows up in `v_unanswered_posts`
`v_moderation_queue`	No rows, the flag was handled
Flag status	actioned

Negative tests

The comment block at the very end holds four statements that should fail. Uncomment one at a time, run it, and confirm the error:

Test	Expected error
Same user votes twice on a post	ORA-00001 (pk_post_vote)
Helpful reply from a different post	ORA-02291 (fk_post_helpful_reply)

Test	Expected error
Second action on the same flag	ORA-00001 (uq_mod_flag)
Pinned post with no pinned_by	ORA-02290 (ck_post_pin_pair)

Self-votes are not on this list on purpose. The database allows them now, so a Java unit test has to cover that rule.

Delete section 8 before the schema goes to the integration environment.

Common errors and integration notes

Error	What it means	Fix
ORA-00001	Duplicate key: a second vote, a repeated name, or a second action on a flag	Check for an existing row first, or catch it and return 409
ORA-02291	Foreign key points at a row that doesn't exist, or a reply on the wrong post	Confirm the parent row exists and belongs to the same post
ORA-02292	Tried to delete a row other rows still point at	Use soft delete (`status = 'removed'`) instead
ORA-02290	A check constraint failed, like a bad status value	Check the allowed values in the table walkthrough
ORA-00955	Object already exists	Run the whole script with F5 so section 0 drops it first
ORA-01031	Not enough privileges	Your user needs CREATE TABLE and CREATE VIEW

The API should turn these into the Shared Core error format, `{"error", "code", "detail"}` , never a raw ORA message.

Before integration

- [ ] Confirm Shared Core is using Oracle. If they pick PostgreSQL, the tables port easily, but types like `VARCHAR2` , `CLOB` , and `NUMBER(1)` change.

- [ ] Remove section 1 and point our foreign keys at Shared Core's user table and ID type.
- [ ] Delete section 8.
- [ ] Post any schema change to the project wiki within 48 hours, per the SOW.